

BANCO DE DESARROLLO PRODUCTIVO

Sistema de Análisis de Microcrédito (S.A.M.)

PRUEBAS UNITARIAS

Evidencia de Cobertura y Código de Calidad Automatizado

Sprints 1 y 2

Versión: 5.0 (Final Consolidado)

Fecha: 09 de junio de 2026

Estado: Reporte de Avance

Equipo de Desarrollo:

Maializ Mamani Quispe

Juan Daniel Ancieta Toledo

Christian Fabian Gonzales Mamani

Leonardo Radek Condori Yucra

Diego Mendez

Clasificación: Uso Interno — Confidencial

1. RESUMEN EJECUTIVO

Este documento consolida el trabajo de aseguramiento de calidad realizado por el equipo de desarrollo durante los Sprints 1 y 2 del proyecto S.A.M. Se implementaron un total de 35 pruebas (25 de backend y 10 de frontend) sobre los 4 módulos correspondientes a los requerimientos funcionales RF-AUTH-01, RF-02.1, RF-02.2 y RF-03.1.

Todas las pruebas han sido ejecutadas exitosamente, alcanzando una cobertura de línea global del 88.3% y una cobertura de ramas del 83.3%. Se mitigó la vulnerabilidad CVE-2024-30172 y se refactorizaron componentes clave para cumplir con los principios de Clean Code y DDD.

Métrica	Valor
Total de pruebas implementadas	35 (25 backend + 10 frontend)
Módulos cubiertos	4 (RF-AUTH-01, RF-02.1, RF-02.2, RF-03.1)
Cobertura de línea global	88.3%
Cobertura de ramas global	83.3%
Vulnerabilidades activas	0 (CVE-2024-30172 mitigada)
Bugs SonarQube	0

2. MARCO TEÓRICO APLICADO Y ARQUITECTURA DE AISLAMIENTO

2.1 Principios Soberanos de Calidad (SDLC)

- Aislamiento Total: Ninguna prueba unitaria interactúa con bases de datos relacionales, discos locales o servidores externos. Toda dependencia compleja (Axon Server, repositorios JPA, APIs de terceros) es suplantada mediante AggregateTestFixture nativo de Axon Framework, garantizando un entorno determinista y repetible.
- Economía del Software: Al validar las reglas lógicas en la unidad mínima de código, el costo de mitigación de bugs se reduce en un factor de 10x respecto a la fase de integración y 100x en comparación con un error descubierto en producción.
- Integración Continua (CI/CD) con Feedback Inmediato: Las pruebas se ejecutan en milisegundos en cada commit. El pipeline automatizado bloquea cualquier compilación defectuosa si un solo caso de prueba falla.

2.2 Estructura del Patrón AAA

Los métodos expuestos aplican estrictamente la secuencia estructurada:

- Arrange — Preparar el escenario, instanciar el AggregateTestFixture y definir el comando.
- Act — Ejecutar el comando a través del CommandHandler del agregado.

- Assert — Verificar mediante afirmaciones que el resultado o excepción obtenida coincide exactamente con la regla de negocio esperada.

2.3 Herramientas Utilizadas

Herramienta	Versión	Propósito
JUnit 5	5.10	Framework de pruebas unitarias
Axon Test	4.9.0	Fixture para agregados CQRS
Jest	29	Pruebas de componentes React
React Testing Library	14	Renderizado y verificación de UI
SonarQube	9.9 LTS	Análisis estático de código
OWASP Dependency-Check	8.4.0	Análisis de vulnerabilidades en dependencias

3. METODOLOGÍA DE TRABAJO EN EQUIPO

- Planificación Conjunta: Antes de escribir código, cada integrante analizó sus requerimientos funcionales y no funcionales asignados, identificando los casos de prueba críticos y los posibles puntos de fallo.
- Desarrollo Guiado por Pruebas (TDD Adaptado): Las pruebas unitarias se diseñaron inmediatamente después de definir la interfaz del agregado o componente, forzando pensar en el contrato de la clase antes de su implementación completa.
- Propiedad Colectiva y Revisión Cruzada: Cada línea de código y cada prueba fueron revisadas por al menos otro miembro del equipo antes de integrarse a la rama principal.
- Aislamiento y Determinismo: Uso estricto de AggregateTestFixture de Axon para las pruebas de dominio, garantizando que las pruebas no dependieran de servicios externos y fueran 100% repetibles.
- Integración Continua (Local): Cada integrante ejecutó todas las pruebas del módulo con `./mvnw.cmd test` antes de cada commit.

4. INFORMES INDIVIDUALES DE PRUEBAS

4.1. MAIALIZ MAMANI QUISPE — RF-AUTH-01 (Pantalla Principal)

4.1.1. Introducción y Objetivo del Módulo

Responsabilidad: Asegurar la calidad del módulo RF-AUTH-01, la puerta de entrada al sistema S.A.M. El objetivo es mostrar a los analistas un menú con todos los modelos de evaluación disponibles, permitiendo una navegación clara y segura según el rol del usuario, cumpliendo con el RNF-05 (Consistencia Visual BDP).

4.1.2. Pruebas Ejecutadas

#	Prueba	Tipo	Resultado
1	Retornar 3 modelos para usuario autenticado	Backend	✓ PASÓ
2	Lanzar excepción si usuario vacío	Backend	✓ PASÓ
3	Lanzar excepción si usuario nulo	Backend	✓ PASÓ
4	Mostrar 3 modelos en pantalla	Frontend	✓ PASÓ
5	Verificar color corporativo #003366	Frontend	✓ PASÓ
6	Verificar íconos por modelo	Frontend	✓ PASÓ
7	Verificar navegación al hacer clic	Frontend	✓ PASÓ

4.1.3. Código de Pruebas Implementado

Backend: ModeloQueryHandlerTest.java

```
package bo.gob.bdp.sam.core.application.query;

import bo.gob.bdp.sam.core.domain.exception.UsuarioNoAutenticadoException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

class ModeloQueryHandlerTest {
    private ModeloQueryHandler queryHandler;

    @BeforeEach
    void setUp() {
        this.queryHandler = new ModeloQueryHandler();
    }

    @Test
    @DisplayName("CP-AUTH-002 // Exito // Debe retornar 3 modelos para un analista autenticado")
    void givenValidAnalyst_whenQueryingModels_thenReturnThreeModels() {
        String usuarioId = "USR-MAMANI";
        String rol = "ROL_ANALISTA_CREDITO";
        List<String> modelos = queryHandler.obtenerModelosDisponibles(usuarioId, rol);
        assertNotNull(modelos, "La lista de modelos no deberia ser nula");
        assertEquals(3, modelos.size(), "Debe retornar exactamente los 3 modelos del BDP");
        assertTrue(modelos.get(0).contains("Agricola"));
        assertTrue(modelos.get(1).contains("Pecuaria"));
        assertTrue(modelos.get(2).contains("Produccion"));
    }

    @Test
```

```
@DisplayName("CP-AUTH-002 // Error // Debe lanzar excepcion si el ID de usuario esta vacio")
void givenEmptyUser_whenQueryingModels_thenThrowUsuarioNoAutenticadoException() {
    UsuarioNoAutenticadoException ex = assertThrows(
        UsuarioNoAutenticadoException.class, () -> {
            queryHandler.obtenerModelosDisponibles("", "ROL_ANALISTA_CREDITO");
        });
    assertEquals("El ID de usuario no puede estar vacio", ex.getMessage());
}
}
```

Frontend: MenuComponent.test.js

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import '@testing-library/jest-dom';
import MenuComponent from './MenuComponent';

describe('Pruebas Unitarias UI - Menu Principal (CP-AUTH-001 / RNF-05)', () => {
    const modelosMock = [
        { id: 'M-01', nombre: 'Evaluacion Agricola' },
        { id: 'M-02', nombre: 'Evaluacion Pecuaria' },
        { id: 'M-03', nombre: 'Evaluacion Produccion' }
    ];

    test('Deberia desplegar los 3 modelos de credito y aplicar el color azul corporativo BDP', () => {
        render(<MenuComponent modelos={modelosMock} />);
        expect(screen.getByText('Evaluacion Agricola')).toBeInTheDocument();
        expect(screen.getByText('Evaluacion Pecuaria')).toBeInTheDocument();
        expect(screen.getByText('Evaluacion Produccion')).toBeInTheDocument();
        const topBanner = screen.getByTestId('bdp-nav-header');
        expect(topBanner).toHaveStyle({ backgroundColor: '#003366' });
    });
});
```

4.1.4. Problemas Encontrados y Soluciones

Problema: El `ModelosCargadosEvent` contiene un `timestamp` que se genera con `Instant.now()`, lo que hacía imposible la comparación directa en las pruebas.

Solución: Se cambió la verificación del evento para usar `expectEventsMatching()` en lugar de `expectEvents()`, validando solo el tipo de evento y los campos relevantes, no el `timestamp`.

4.1.5. Conclusión

El módulo RF-AUTH-01 funciona correctamente. El menú muestra los 3 modelos requeridos, valida usuarios no autenticados y cumple con el estándar visual del BDP (RNF-05). Todas las pruebas pasaron sin errores.

Métrica	Valor
Cobertura de línea	91.0%
Cobertura de ramas	85.7%

4.2. JUAN DANIEL ANCIETA TOLEDO — RF-02.1 (Registro de Cliente)

4.2.1. Introducción y Objetivo del Módulo

Módulo RF-02.1, Registro de Cliente. Este componente es fundamental porque constituye la creación del expediente digital único del prestatario. El objetivo era garantizar que el sistema valide rigurosamente la información, evite registros duplicados y sienta las bases para la futura auditoría.

4.2.2. Pruebas Ejecutadas

#	Prueba	Tipo	Resultado
1	Registrar cliente con datos válidos	Backend	✓ PASÓ
2	Rechazar CI menor a 7 dígitos	Backend	✓ PASÓ
3	Rechazar nombre vacío	Backend	✓ PASÓ
4	Rechazar nombre nulo	Backend	✓ PASÓ
5	Reconstruir estado desde evento	Backend	✓ PASÓ
6	Mostrar error si CI duplicado (400)	Frontend	✓ PASÓ
7	Mostrar error si fallo interno (500)	Frontend	✓ PASÓ
8	Mostrar error si timeout de red	Frontend	✓ PASÓ

4.2.3. Código de Pruebas Implementado

Backend: ClienteAggregateTest.java

```
class ClienteAggregateTest {
    private FixtureConfiguration<ClienteAggregate> fixture;

    @BeforeEach
    void setUp() {
        fixture = new AggregateTestFixture<>(ClienteAggregate.class);
    }

    @Test
    @DisplayName("CP-REG-001 // Exito // Registrar nuevo cliente con CI y nombre validos")
    void givenValidClientData_whenRegistering_thenExpedientIsCreated() {
        RegistrarClienteCommand cmd = new RegistrarClienteCommand(
            "4782910", "JUAN DANIEL ANCIETA", "juan@email.com", "77777777"
        );
        fixture.givenNoPriorActivity().when(cmd).expectSuccessfulHandlerExecution();
    }
}
```

```

    }

    @Test
    @DisplayName("CP-REG-002 // Restriccion // Rechazar CI menores a 7 digitos")
    void givenShortDocumentId_whenRegistering_thenThrowDocumentoInvalidoException() {
        RegistrarClienteCommand cmdCorto = new RegistrarClienteCommand(
            "99122", "L. Radek", "radek@email.com", "77777777"
        );
        fixture.givenNoPriorActivity().when(cmdCorto)
            .expectException(DocumentoInvalidoException.class);
    }
}

```

4.2.4. Problemas Encontrados y Soluciones

- Problema: El @EventSourcingHandler no tenía prueba explícita. Solución: Se agregó prueba con fixture.given(evento).when(comando).expectState() para validar la reconstrucción del estado desde el Event Store.
- Problema: La validación de duplicados se realiza en un HashMap en memoria. Solución: Documentado el riesgo. Migración a PostgreSQL planificada para Sprint 3.

4.2.5. Conclusión

El módulo RF-02.1 funciona correctamente. La validación de longitud de CI (mínimo 7 dígitos) opera según lo especificado. Las pruebas de error del frontend cubren los escenarios 400, 500 y timeout.

Métrica	Valor
Cobertura de línea	92.5%
Cobertura de ramas	87.3%

4.3. CHRISTIAN FABIAN GONZALES MAMANI — RF-02.2 (Checklist de Documentos)

4.3.1. Introducción y Objetivo del Módulo

Módulo RF-02.2, Checklist de Documentos. Este componente actúa como un guardián de la calidad y la legalidad del proceso crediticio. Su función es asegurar que la carpeta del cliente cumpla con la normativa antes de permitir el avance a la evaluación financiera, bloqueando la transición si faltan documentos obligatorios.

4.3.2. Pruebas Ejecutadas

#	Prueba	Tipo	Resultado
1	Checklist con todos los obligatorios	Backend	✅ PASÓ
2	Bloquear si falta obligatorio y bloquearSiIncompleto=true	Backend	✅ PASÓ

#	Prueba	Tipo	Resultado
3	Verificar mensaje exacto de la excepción	Backend	✓ PASÓ
4	Permitir avance si solo faltan opcionales	Backend	✓ PASÓ
5	Rechazar cliente nulo	Backend	✓ PASÓ
6	Reconstruir estado desde evento	Backend	✓ PASÓ
7	Botón 'Continuar' deshabilitado si faltan obligatorios	Frontend	✓ PASÓ
8	Botón 'Continuar' habilitado si todos marcados	Frontend	✓ PASÓ

4.3.3. Código de Pruebas Implementado

Backend: DocumentoAggregateTest.java

```
class DocumentoAggregateTest {
    private FixtureConfiguration<DocumentoAggregate> fixture;

    @Test
    @DisplayName("CP-CHK-001 // Regla de Negocio // Bloquear si falta documento obligatorio")
    void givenMissingRequiredDocument_whenFinalValidation_thenThrowException() {
        Map<String, Boolean> docs = new HashMap<>();
        docs.put("CI_FRENTE", true); docs.put("CI_REVERSO", true);
        docs.put("FACTURA_AGUA", false); docs.put("FACTURA_LUZ", true);
        docs.put("ESTADO_CUENTA", true);
        ActualizarChecklistCommand cmd = new ActualizarChecklistCommand("12345678", docs,
true);
        fixture.givenNoPriorActivity().when(cmd)
            .expectException(DocumentacionIncompletaException.class);
    }

    @Test
    @DisplayName("CP-CHK-002 // Regla de Negocio // Permitir avance si solo faltan opcionales")
    void givenOnlyOptionalDocumentsMissing_whenFinalValidation_thenAllowTransition() {
        Map<String, Boolean> docs = new HashMap<>();
        docs.put("CI_FRENTE", true); docs.put("CI_REVERSO", true);
        docs.put("FACTURA_AGUA", true); docs.put("FACTURA_LUZ", true);
        docs.put("ESTADO_CUENTA", true);
        docs.put("AVAL_BANCARIO", false); docs.put("DECLARACION_JURADA", false);
        ActualizarChecklistCommand cmd = new ActualizarChecklistCommand("12345678", docs,
true);
        fixture.givenNoPriorActivity().when(cmd).expectSuccessfulHandlerExecution();
    }
}
```

4.3.4. Problemas Encontrados y Soluciones

- Problema: La validación de bloqueo solo existía en el frontend. Solución: Se implementó la validación en el @CommandHandler del agregado, creando

DocumentacionIncompletaException. Esto garantiza que la regla de negocio se cumpla en ambos lados.

- Problema: El campo se llamaba validacionFinal, poco descriptivo. Solución: Refactorizado a bloquearSilncompleto.
- Problema: La dependencia bcprov-jdk15on 1.78 presentaba CVE de severidad media. Solución: Actualizada a versión 1.79, mitigando CVE-2024-30172.

4.3.5. Conclusión

El módulo RF-02.2 es ahora un guardián robusto y seguro. La validación de documentos obligatorios funciona tanto en el frontend como en el backend.

Métrica	Valor
Cobertura de línea	91.8%
Cobertura de ramas	86.1%

4.4. LEONARDO RADEK CONDORI YUCRA — RF-03.1 (Volteo de Balances)

4.4.1. Introducción y Objetivo del Módulo

Módulo más crítico: el Volteo de Balances (RF-03.1). Este módulo es el corazón de la estandarización contable. Su objetivo es validar la consistencia de la ecuación Activo = Pasivo + Patrimonio con una tolerancia de ±10 Bolivianos (RNF-02). Un error aquí podría resultar en una evaluación de riesgo incorrecta.

4.4.2. Pruebas Ejecutadas

#	Escenario	Diferencia	Resultado
1	Balance cuadrado exacto	0 Bs	✔ Aceptado
2	Dentro del margen	5 Bs	✔ Aceptado
3	Límite exacto	10 Bs	✔ Aceptado
4	Excede por decimales	10.05 Bs	✖ Rechazado
5	Descuadre severo	2000 Bs	✖ Rechazado
6	Reconstruir estado desde evento	—	✔ PASÓ

4.4.3. Código de Pruebas Implementado

Backend: BalanceAggregateTest.java (Prueba Parametrizada)

```
@ParameterizedTest(name = "Escenario: Act={0}, Pas+Pat={1} -> Espera Aceptacion={2}")
@CsvSource({
    "250000.00, 250000.00, true, Diferencia exacta de 0 Bs (Balance perfecto)",
    "250005.00, 250000.00, true, Diferencia de 5 Bs (Dentro del margen)",
    "250010.00, 250000.00, true, Diferencia limite de 10 Bs (Umbral maximo)",
```

```

        "250010.05, 250000.00, false, Diferencia de 10.05 Bs (Rechazado)",
        "252000.00, 250000.00, false, Diferencia severa de 2000 Bs (Rechazo absoluto)"
    })
    @DisplayName("CP-RNF02-001 // Parametrizado // Validar margen estricto de descuadre +/- 10 Bs")
    void validateBalanceToleranceThresholds(String activoStr, String pasivoPatStr,
        boolean expectedResult, String descripcion) {
        Map<String, BigDecimal> cuentas = new HashMap<>();
        cuentas.put("CAJA_BANCOS", new BigDecimal(activoStr));
        cuentas.put("CAPITAL_SOCIAL", new BigDecimal(pasivoPatStr));
        ActualizarBalanceCommand cmd = new ActualizarBalanceCommand(
            "12345678", cuentas, expectedResult);
        if (expectedResult) {
            fixture.givenNoPriorActivity().when(cmd).expectSuccessfulHandlerExecution();
        } else {
            fixture.givenNoPriorActivity().when(cmd)
                .expectException(BalanceDescuadradoException.class);
        }
    }
}

```

4.4.4. Problemas Encontrados y Soluciones

- Problema: Los cálculos usaban double, con posible pérdida de precisión en montos financieros. Solución: Refactorizado el BalanceAggregate para usar java.math.BigDecimal con comparaciones .compareTo().
- Problema: El campo validacionFinal era ambiguo. Solución: Renombrado a bloquearSiDescuadrado.

4.4.5. Conclusión

El módulo RF-03.1 está validado y cumple estrictamente con el RNF-02. La prueba parametrizada cubre los 5 escenarios del margen ± 10 Bs. El uso de BigDecimal garantiza precisión en los cálculos financieros.

Métrica	Valor
Cobertura de línea	94.2%
Cobertura de ramas	91.5%

4.5. DIEGO MENDEZ — Pruebas de Aceptación

4.5.1. Introducción y Objetivo

Como Product Owner, la responsabilidad fue verificar que los 4 módulos implementados cumplan con los criterios de aceptación definidos en el contrato. Se realizaron pruebas manuales desde el navegador siguiendo el flujo completo del usuario.

4.5.2. Pruebas de Aceptación Ejecutadas

#	RF	Flujo Probado	Resultado
1	RF-AUTH-01	Navegar a / y verificar 3 modelos con colores BDP	✓ Aceptado
2	RF-02.1	Registrar cliente con datos válidos y verificar redirección	✓ Aceptado
3	RF-02.1	Registrar cliente duplicado y verificar mensaje de error	✓ Aceptado
4	RF-02.2	Completar checklist y verificar botón habilitado	✓ Aceptado
5	RF-03.1	Ingresar balance descuadrado >10 Bs y verificar rechazo	✓ Aceptado
6	RF-03.1	Ingresar balance dentro del margen ≤10 Bs y verificar aceptación	✓ Aceptado

4.5.3. Observaciones

- El flujo completo funciona: Registro → Checklist → Volteo de Balances.
- Los colores corporativos (#003366, #FFC107) se aplican correctamente.
- Las validaciones en tiempo real mejoran la experiencia de usuario.
- Se recomienda agregar un indicador de carga en el botón 'Crear Expediente' y migrar los alert() a componentes Snackbar de MUI.

4.5.4. Conclusión

Los 4 módulos cumplen con los criterios de aceptación. El sistema es funcional y navegable. Se aprueba el incremento de los Sprints 1 y 2.

5. MÉTRICAS CONSOLIDADAS

5.1. Cobertura por Archivo

Archivo de Prueba	Línea	Ramas	Responsable
ModeloQueryHandlerTest.java	91.0%	85.7%	Maializ Mamani
ClienteAggregateTest.java	92.5%	87.3%	Juan Daniel Ancieta
DocumentoAggregateTest.java	91.8%	86.1%	Christian Gonzales
BalanceAggregateTest.java	94.2%	91.5%	Leonardo Radek
MenuComponent.test.js	80.0%	75.0%	Maializ Mamani
ChecklistComponent.test.js	80.2%	75.0%	Christian Gonzales
RegistroCliente.test.js	88.5%	82.4%	Juan Daniel Ancieta
CONSOLIDADO GLOBAL	88.3%	83.3%	Todo el equipo

5.2. Análisis de Vulnerabilidades

Herramienta	Bugs	Vulnerabilidades	Code Smells
SonarQube 9.9 LTS	0	0	3
OWASP Dependency-Check 8.4.0	—	0 (CVE-2024-30172 mitigado)	—

5.3. Justificación del Código No Cubierto

Clase	% No Cubierto	Motivo	Plan
ModeloQueryHandler	9.0%	Método obtenerModelosPorRol() requiere RF-07.1	Sprint 6
ClienteAggregate	7.5%	@EventSourcingHandler en prueba indirecta	Sprint 3
DocumentoAggregate	8.2%	@EventSourcingHandler en prueba indirecta	Sprint 3
BalanceAggregate	5.8%	Métodos privados de cálculo	Sprint 3
Componentes React	15-20%	Ramas de error de red	Sprint 3

6. HALLAZGOS, CORRECCIONES Y DEUDA TÉCNICA

#	Hallazgo	Severidad	Responsable	Solución
1	Lógica de consulta en Command Side (RF-AUTH-01)	Media	Maializ	Refactorizado a ModeloQueryHandler (Query Side)
2	Validación de Checklist solo en Frontend (RF-02.2)	Crítica	Christian	Implementada DocumentacionIncompletaException en backend
3	Uso de double en cálculos financieros (RF-03.1)	Crítica	Leonardo	Refactorizado a BigDecimal
4	Nomenclatura ambigua validacionFinal	Baja	Christian / Leonardo	Renombrado a bloquearSiIncompleto y bloquearSiDescuadrado
5	Query Side en HashMap (Pérdida de datos)	Alta	Juan Daniel	Documentado. Migrar a PostgreSQL en Sprint 3
6	Vulnerabilidad CVE-2024-30172	Media	Christian	Actualizado bcprov-jdk15on a versión 1.79

7. TRAZABILIDAD REQUERIMIENTOS ↔ PRUEBAS

ID Prueba	RF Cubierto	RNF Cubierto	# Pruebas	Responsable
CP-AUTH-001/002	RF-AUTH-01	RNF-05	7	Maializ Mamani
CP-REG-001 a 006	RF-02.1	—	8	Juan Daniel Ancieta
CP-CHK-001 a 004	RF-02.2	—	8	Christian Gonzales
CP-RNF02-001	RF-03.1	RNF-02	6	Leonardo Radek
Pruebas de Aceptación	RF-01 a 03	RNF-05	6	Diego Mendez

8. CATÁLOGO DE EXCEPCIONES PERSONALIZADAS DE DOMINIO

Excepción	Propósito	Agregado
UsuarioNoAutenticadoException	Usuario no autenticado o inválido	ModeloQueryHandler
DocumentoInvalidoException	CI/NIT con formato incorrecto	ClienteAggregate
ClienteNoEncontradoException	Cliente nulo o no existente	ClienteAggregate, DocumentoAggregate, BalanceAggregate
DocumentacionIncompletaException	Faltan documentos obligatorios	DocumentoAggregate
BalanceDescuadradoException	Descuadre contable superior a ± 10 Bs	BalanceAggregate

9. COMPROMISOS PARA SPRINT 3

Tarea	Responsable
Migrar Query Side a PostgreSQL	Juan Daniel Ancieta
Agregar CircularProgress en botones y unificar notificaciones con Snackbar MUI	Maializ Mamani
Pruebas de integración backend-frontend	Christian Gonzales
Implementar y probar RF-03.2 Indicadores Financieros	Leonardo Radek
Revisar Code Smells pendientes	Todo el equipo
Alcanzar cobertura de ramas $\geq 85\%$	Todo el equipo

— *Fin del Documento* — Manual de Ingeniería S.A.M. v5.0 | 09 de junio de 2026 —